Sub-Terrain Challenge: Autonomous Cave Exploration with a Quadrotor UAV

Group 16 – Autonomous Systems WS25/26

Yujia Wang[1], Ziming Liu[1], Jiacheng Li[1], Nihat Akın[1], and Iliasu Olayiwola Salaudeen[1]

[1]Technical University of Munich, Munich, Germany.

March 3, 2026

Abstract

We describe the design and implementation of our autonomous quadrotor system for the Sub-Terrain Challenge. The drone is tasked with exploring a simulated cave, building a 3D voxel map, and locating four lantern objects as fast as possible. Our pipeline runs on ROS 2 Jazzy and connects to an instructor-provided Unity simulation through a TCP/UDP bridge. Depth images are converted to point clouds and fused into an OctoMap occupancy grid. An A* planner generates collision-free paths, which are smoothed into time-parameterized trajectories with a trapezoidal velocity profile. An SE(3) geometric controller, also provided with the course material, tracks the trajectory and sends rotor commands back to the simulator. Lantern detection is achieved through semantic segmentation fused with depth data.

1 Introduction

The Sub-Terrain Challenge requires an autonomous UAV to enter a cave environment, find four lantern objects, report their positions, and produce a 3D representation of the explored space. The entire system runs inside a Unity-based simulation, interfaced through ROS 2. The course provided several base packages—the Unity simulation bridge, a geometric SE(3) flight controller, and MAV message definitions—on top of which we built our own perception, detection, planning, mapping, and mission-management modules. In total the workspace contains 14 ROS 2 packages. Our main design goal was to keep our own packages independent so that team members could develop and test in parallel. This report gives an overview of the architecture, describes the algorithms we implemented, and documents who contributed what.

2 System Architecture

The system begins at the simulation bridge, which receives raw sensor data from Unity via TCP and publishes it as standard ROS 2 messages. Perception nodes convert depth images into 3D point clouds and then into an OctoMap occupancy grid. A separate detection node finds lanterns by fusing the semantic camera with depth. The path planner reads the occupancy grid and the current goal, runs A* search, and outputs a waypoint path. The trajectory planner turns this path into a smooth, time-parameterized trajectory. Finally, the SE(3) controller computes rotor speed commands, which are sent back to Unity over UDP.

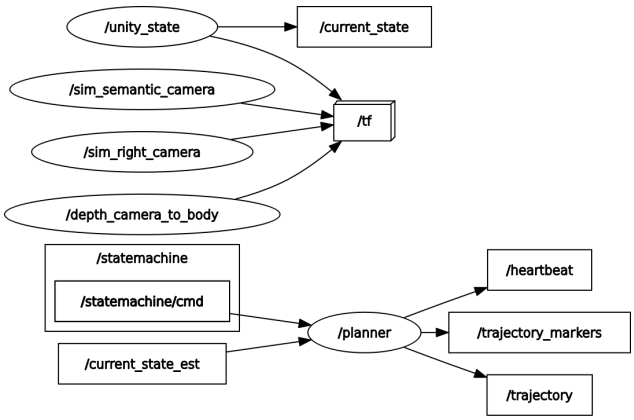

Figure 1: ROS computation graph during the Subterrain Challenge runtime.

2.1 Simulation Interface (Instructor-Provided)

The simulation infrastructure was provided by the course staff. The `simulation` package handles all communication with the Unity simulator. A TCP client (`unity_ros`) connects to Unity, reads binary packets carrying sensor data (RGB, depth, IMU, ground-truth state), and publishes them as standard ROS 2 messages. A UDP sender (`w_to_unity`) streams rotor speed commands back at 1 kHz. A `state_estimate_corruptor` node injects Gaussian noise and random-walk drift into the ground-truth pose, publishing corrupted odometry on `/current_state_est` to simulate realistic sensor imperfections. The parsers also handle the left-handed (Unity) to right-handed (ROS) coordinate conversion.

The `controller_pkg`, also instructor-provided, implements an SE(3) geometric tracking controller [1] as a precompiled library. It subscribes to desired trajectory commands and the estimated state, and outputs `mav_msgs/Actuators` messages containing the four rotor angular velocities. The MAV message definitions (`mav_msgs`, `mav_planning_msgs`) were ported from the ETH-ASL `mav_comm` repository [2].

2.2 Perception

The perception pipeline consists of two cascaded nodes.

Depth to point cloud. The `depth_to_pointcloud` node subscribes to the depth camera image (`16UC1`, millimeters) and the corresponding `CameraInfo`. For each pixel it applies the standard pinhole back-projection:

$$x = \frac{(u - c_x)\, z}{f_x}, \quad y = \frac{(v - c_y)\, z}{f_y} \tag{1}$$

where (c_x, c_y) is the principal point and (f_x, f_y) the focal lengths. A configurable downsample factor and a maximum-depth threshold reduce the output size.

Point cloud to OctoMap. The `pointcloud_to_octomap` node receives the point cloud, transforms each point from the camera frame to the world frame via TF2, and inserts them into an OctoMap octree [3]. We use a resolution of 0.15 m, with hit probability 0.65, miss probability 0.2, and clamping thresholds of 0.12 and 0.97. The resulting occupancy grid is published as `octomap_msgs/Octomap` and additionally as a colour-coded `MarkerArray` for visualization in RViz2.

2.3 Lantern Detection

The detection node synchronizes the semantic camera image with the depth image using an approximate-time message filter. It converts the semantic image to grayscale, applies a threshold (intensity > 10) to obtain a binary mask, and back-projects the masked pixels to 3D using the same pinhole model. After transforming to the world frame, the resulting 3D position is compared against a running list of known lanterns. If a new detection falls within 1.5 m of an existing entry, the position is updated as a weighted average:

$$\mathbf{p}_{\text{new}} = \frac{n\,\mathbf{p}_{\text{old}} + \mathbf{p}_{\text{det}}}{n+1} \tag{2}$$

Otherwise, a new lantern is registered. Detected lanterns are published as both a `PoseArray` and a yellow `MarkerArray` for debugging.

2.4 Path Planning

Our path planner implements the A* algorithm on a 3D voxel grid [4]. When a new OctoMap message arrives, the node iterates over all occupied leaf nodes and inflates them by a configurable safety margin (0.5 m in every direction) to account for the drone's body size. Occupied voxels are stored in a hash set keyed by a 64-bit integer that encodes three 21-bit signed coordinates, giving $O(1)$ collision checks.

The A* search uses a 26-connected neighbourhood (including diagonals) with a Euclidean heuristic. We cap the number of expanded nodes at 100 000 to keep planning times bounded. After the search, the resulting path is decimated by keeping every third waypoint, which avoids feeding an unnecessarily dense sequence to the trajectory planner. If no map data is available yet or A* fails to find a path, the planner falls back to a simple 20-point straight-line interpolation toward the goal.

2.5 Trajectory Planning

The trajectory planner converts the waypoint path into a time-parameterized `MultiDOFJointTrajectory` using a trapezoidal velocity profile. Given a desired cruise speed $v_{\max}$ and maximum acceleration $a_{\max}$, it

first computes the acceleration distance:

$$d_{\text{accel}} = \frac{v_{\max}^2}{2\,a_{\max}} \tag{3}$$

If the total path length is too short to reach cruise speed, the acceleration distance is halved. Each waypoint is then assigned a speed depending on its phase:

- **Acceleration:** $v = \max\!\left(\sqrt{2\,a_{\max}\,d},\ 0.1\right)$
- **Cruise:** $v = v_{\max}$
- **Deceleration:** $v = \max\!\left(\sqrt{2\,a_{\max}\,d_{\text{rem}}},\ 0.1\right)$

The velocity vector is aligned with the direction to the next waypoint. Inter-waypoint time intervals are computed as $\Delta t = d_{\text{seg}}/v_{\text{avg}}$, clipped to a minimum of 0.05 s. The last point receives zero velocity so the drone comes to a full stop at the goal. We use $v_{\max} = 2.0\,\text{m/s}$ and $a_{\max} = 1.5\,\text{m/s}^2$ for a stable flight profile.

2.6 State Machine

The mission state machine manages the high-level flight sequence. It cycles through the following states:

1. **INIT** – wait for the first valid odometry and record the home position.
2. **TAKEOFF** – ascend to 10 m altitude.
3. **TO_CAVE_ENTRANCE** – follow a predefined set of waypoints to reach the cave opening.
4. **EXPLORE** – autonomously explore the cave using frontier-based goals from the mapping module; detect lanterns along the way.
5. **RETURN_HOME** – navigate back to the recorded home position.
6. **LAND** – descend to 0.4 m and stop.
7. **COMPLETE** – mission finished.

A 200 ms tick timer drives transitions. The exploration phase terminates either when all four lanterns are found or after a 7-minute timeout. In the exploration state, the node subscribes to frontier goals published by the mapping module and forwards them as `PoseStamped` goals to the path planner. An arrival tolerance of 5 m triggers the transition to the next waypoint or state.

2.7 Mapping

The mapping package was designed to maintain a persistent voxel-grid representation alongside frontier-based exploration goals [5]. Mapping responsibilities are split across mapping and mapping_pkg. In mapping, nodes such as voxel_grid_mapper_node and global_map_publisher_node maintain voxel/global map products and expose map_related outputs/services. In mapping_pkg, octomap_server_node builds occupancy maps from point cloud input for planning.

2.8 Custom Messages and Services

The `utils` package defines the shared message and service types used across the system:

- `LanternPose` / `LanternPoseArray`: detected lantern positions with confidence and confirmation fields.
- `GlobalMap`: full voxel-grid map with occupied, free, and frontier data plus metadata (resolution, explored volume, storage path).
- `SaveMap.srv`: persists the current map to disk.
- `SetMissionMode.srv`: allows manual override of the active FSM state.

3 Design Choices

ROS 2 Jazzy on Ubuntu 24.04. ROS 2 Jazzy [6] was chosen for its long-term support status and availability on Ubuntu 24.04. All inter-node communication uses standard topics; services are reserved for on-demand operations like saving the map or overriding the mission mode.

Modular package layout. Each functional block lives in its own ROS 2 package with a dedicated launch file, which let us develop and test components independently. A master launch file in the `utils` package can bring everything up at once.

OctoMap for 3D occupancy. We used the OctoMap library [3, 7] rather than a flat voxel array because its octree compression reduces memory usage for large cave environments while still supporting ray casting and probabilistic updates.

A* path planning. We considered RRT and its variants, but A* on a discretised voxel grid offers deterministic, resolution-complete paths with straightforward obstacle inflation. With a 0.15 m resolution and a 100 k node expansion budget, planning finishes well within real-time constraints.

Trapezoidal velocity profile. A polynomial trajectory (e.g. minimum-snap) would produce smoother motion, but the trapezoidal profile is simpler to implement, easier to tune, and gives predictable acceleration/deceleration phases that are sufficient for cave navigation at 2 m/s cruise speed.

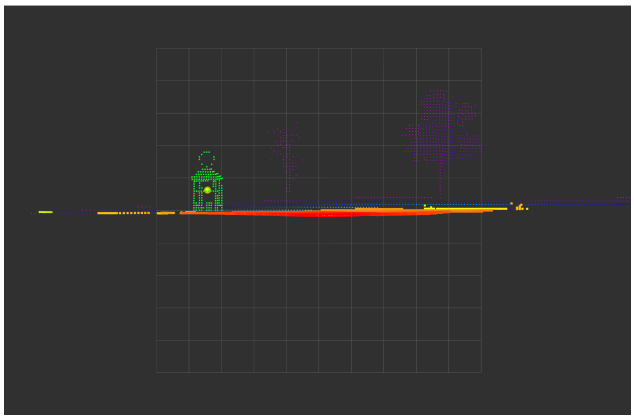

Figure 2: Visualization in RViz2 showing the OctoMap occupancy grid (coloured by height), the drone's estimated pose (TF frame), and detected lanterns (yellow spheres).

4 Limitations and Known Issues

Several modules are architecturally designed but not yet fully implemented. The `mapping` node (persistent voxel grid and frontier goals), the `state_machine` (mission FSM sequencing), and the `quadrotor_controller` (100 Hz trajectory sampler) all have complete header definitions and parameter files, but their callback bodies remain as TODO placeholders. As a result, the system does not yet autonomously sequence the full mission (take-off, cave entry, exploration, return, landing); navigation currently requires manual goal publication. The `perception_pipeline` package was planned as an extended alternative to the `perception` and `detection` nodes—with semantic mask filtering and `LanternPoseArray` output—but was not completed.

Another limitation is that the OctoMap is rebuilt from scratch every frame rather than accumulated over time, which means the path planner only sees obstacles currently in the field of view. A persistent map would require the mapping module, which is not yet functional.

5 Results

With the perception pipeline running, the system produces dense OctoMap updates and successfully detects lanterns when they enter the semantic camera's field of view. The path planner generates collision-free paths in real time, and the trajectory planner provides smooth velocity profiles that the SE(3) controller tracks reliably. Figure 2 shows a representative RViz2 snapshot during a test run.

6 Code Attribution and Contributions

6.1 External Code

The following components were provided by the course or taken from external sources:

- `simulation` package – complete Unity–ROS 2 bridge (TCP/UDP communication, sensor parsers, state corruptor, TF publishers, launch files). Provided by the course staff.
- `controller_pkg` – SE(3) geometric controller with precompiled `libcontroller_lib.so`. Provided by the course staff [1].
- `mav_msgs` and `mav_planning_msgs` – MAV message definitions ported from ETH-ASL `mav_comm` [2]. Provided with the course material.
- OctoMap library [3] – used via system packages.
- `libsocket` – third-party TCP/UDP socket library, fetched at build time by the simulation package.

6.2 Contribution Matrix

Table 1 summarises who worked on which packages.

Table 1: Package contributions per team member.

Member	Packages / Responsibilities
Iliasu Salaudeen	`state_machine`, `controller_pkg` integration: mission FSM architecture, controller launch configuration
Jiacheng Li	`perception`, `detection`: depth-to-pointcloud, OctoMap integration, lantern detector with semantic+depth fusion
Nihat Akın	`path_planner`, `trajectory_planner`: A* search on inflated voxel grid, trapezoidal velocity profiling, waypoint decimation
Yujia Wang	`mapping`, `utils`, `perception_pipeline`: voxel grid mapper architecture, custom messages/services, master launch file, system integration
Ziming Liu	`mapping`, `quadrotor_controller`: simulation integration, trajectory sampler architecture, system-wide testing

All members contributed to debugging, testing, and the written report.

Acknowledgements

We thank the Autonomous Systems teaching team at TUM for providing the simulation environment, base code, and support throughout the project.

References

1. Lee T, Leok M, and McClamroch NH. Geometric Tracking Control of a Quadrotor UAV on SE(3). 49th IEEE Conference on Decision and Control (CDC) 2010:5420–5.
2. ETH Zurich ASL. mav_comm: MAV communication ROS packages. `https://github.com/ethz-asl/mav_comm`. Accessed: 2026-02-15. 2019.
3. Hornung A, Wurm KM, Bennewitz M, Stachniss C, and Burgard W. OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees. Autonomous Robots 2013;34:189–206.
4. Hart PE, Nilsson NJ, and Raphael B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. In: *IEEE Transactions on Systems Science and Cybernetics*. Vol. 4. 2. 1968:100–7. DOI: `10.1109/TSSC.1968.300136`.
5. Yamauchi B. A Frontier-Based Approach for Autonomous Exploration. Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation 1997:146–51.
6. Macenski S, Foote T, Gerkey B, Lalancette C, and Woodall W. Robot Operating System 2: Design, architecture, and uses in the wild. Science Robotics 2022;7:eabm6074.
7. Hornung A and Wurm KM. OctoMap – 3D occupancy mapping library. `https://octomap.github.io/`. Accessed: 2026-02-10. 2013.